actuarialmath

Terence Lim

PhD ASA CAIA CFA CFP CMA CPA FRM

May 05, 2024

APIS:

This Python package implements fundamental methods for modeling life contingent risks, and closely follows the coverage of traditional topics in actuarial exams and standard texts such as the "Fundamentals of Actuarial Math - Long-term" exam syllabus by the Society of Actuaries, and "Actuarial Mathematics for Life Contingent Risks" by Dickson, Hardy and Waters.

CHAPTER

ONE

INTRODUCTION

1.1 Overview

The package comprises three sets of classes, which:

1. Implement general actuarial methods
    - Basic interest theory and probability laws
    - Survival functions, expected future lifetimes and fractional ages
    - Insurance, annuity, premiums, policy values, and reserves calculations
2. Adjust results for
    - Extra mortality risks
    - 1/mthly payment frequency using UDD or Woolhouse approaches
3. Specify and load a particular form of assumptions
    - Recursion inputs
    - Life table, select life table, or standard ultimate life table
    - Mortality laws, such as constant force of maturity, beta and uniform distributions, or Makeham's and Gompertz's laws

1.2 Quick Start

1. `pip install actuarialmath`
    - also requires *numpy*, *scipy*, *matplotlib* and *pandas*.
2. Start Python (version >= 3.10) or Jupyter-notebook
    - Select a suitable subclass to initialize with your actuarial assumptions, such as *MortalityLaws* (or a special law like *ConstantForce*), *LifeTable*, *SULT*, *SelectLife* or *Recursion*.
    - Call appropriate methods to compute intermediate or final results, or to *solve* parameter values implicitly.
    - Adjust answers with *ExtraRisk* or *Mthly* (or its *UDD* or *Woolhouse*) classes.

1.3 Examples

```
# SOA FAM-L sample question 5.7
from actuarialmath import Recursion, Woolhouse
# initialize Recursion class with actuarial inputs
life = Recursion().set_interest(i=0.04)\
                  .set_A(0.188, x=35)\
                  .set_A(0.498, x=65)\
                  .set_p(0.883, x=35, t=30)
# modfy the standard results with Woolhouse mthly approximation
mthly = Woolhouse(m=2, life=life, three_term=False)
# compute the desired temporary annuity value
print(1000 * mthly.temporary_annuity(35, t=30)) #   solution = 17376.7
```

```
# SOA FAM-L sample question 7.20
from actuarialmath import SULT, Contract
life = SULT()
# compute the required FPT policy value
S = life.FPT_policy_value(35, t=1, b=1000)  # is always 0 in year 1!
# input the given policy contract terms
contract = Contract(benefit=1000,
                    initial_premium=.3,
                    initial_policy=300,
                    renewal_premium=.04,
                    renewal_policy=30)
# compute gross premium using the equivalence principle
G = life.gross_premium(A=life.whole_life_insurance(35), **contract.premium_terms)
# compute the required policy value
R = life.gross_policy_value(35, t=1, contract=contract.set_contract(premium=G))
print(R-S)   # solution = -277.19
```

1.4 Resources

1. Jupyter notebook or run in Colab, to solve all sample SOA FAM-L exam questions
2. User Guide, or download pdf
3. API reference
4. Github repo and issues

CHAPTER

TWO

APIS

2.1 actuarial

Define base class for actuarial math, with utility helpers and constants

MIT License. Copyright (c) 2022-2023 Terence Lim

class actuarialmath.actuarial.**Actuarial**

Bases: object

Define constants and common utility functions

Constants:

VARIANCE : select variance as the statistical moment to calculate

WHOLE : indicates that term of insurance or annuity is Whole Life

VARIANCE = -2

WHOLE = -999

static isclose(*r: float*, *target: float = 0.0*, *abs_tol=1e-06*) → bool

Is close to zero or target value

Parameters

- **r** – value to test if close to zero or target
- **target** – target value, default is 0.0

static integral(*fun: Callable[[float], float]*, *lower: float*, *upper: float*) → float

Compute integral of the function between lower and upper limits

Parameters

- **fun** – function to integrate
- **upper** – upper limit
- **lower** – lower limit

static derivative(*fun: Callable[[float], float]*, *x: float*) → float

Compute derivative of the function at a value

Parameters

- **fun** – function to compute derivative
- **x** – value to compute derivative at

Examples

```
>>> print(Actuarial.derivative(fun=lambda x: x/50, x=25))
```

static solve(*fun: Callable[[float], float]*, *target: float*, *grid: float | Tuple | List*, *mad: bool = False*) → float

Solve root, or parameter that minimizes absolute value, of a function

Parameters

- **fun** – function to compute output given input values
- **target** – target value of function output
- **grid** – initial range of guesses
- **root** – whether to solve root (True), or minimize absolute function (False)

Returns

value s.t. output of function fun(value) ~ target

Examples

```
>>> print(Actuarial.solve(fun=lambda omega: 1/omega,
>>>                       target=0.05, grid=[1, 100]))
```

add_term(*t: int*, *n: int*) → int

Add two terms, either term may be Whole Life

Parameters

- **t** – first term to add
- **n** – second term to add

max_term(*x: int*, *t: int*, *u: int = 0*) → int

Decrease term t if adding deferral period u to (x) exceeds maxage

Parameters

- **x** – age
- **t** – term of insurance or annuity, after deferral period
- **u** – term deferred

Returns

value of term t adjusted by deferral and maxage s.t. maxage not exceeded

2.2 interest

Interest Theory - Applies interest rate formulas

MIT License. Copyright (c) 2022-2023 Terence Lim

class actuarialmath.interest.**Interest**(*i: float = -1.0*, *delta: float = -1.0*, *d: float = -1.0*, *v: float = -1.0*, *i_m: float = -1.0*, *d_m: float = -1.0*, *m: int = 0*, *v_t: Callable[[float], float] | None = None*)

Bases: *Actuarial*

Store an assumed interest rate, and compute interest rate functions

Parameters

- **i** – assumed annual interest rate
- **d** – or annual discount rate
- **v** – or annual discount factor
- **delta** – or continuously compounded interest rate
- **v_t** – or discount rate as a function of time
- **i_m** – or m-thly interest rate
- **d_m** – or m-thly discount rate
- **m** – m'thly frequency, if i_m or d_m are specified

property i: float

effective annual interest rate

property d: float

annual discount rate of interest

property delta: float

continuously compounded interest rate, or force of interest, per year

property v: float

annual discount factor

property v_t: Callable

discount factor as a function of time

annuity(*t: int = -1*, *m: int = 1*, *due: bool = True*) → float

Compute value of the annuity certain factor

Parameters

- **t** – number of years of payments
- **m** – m'thly frequency of payments (0 for continuous payments)
- **due** – whether annuity due (True) or immediate (False)

Examples

```
>>> print(interest.annuity(t=10, due=False), 2.831059)
```

static mthly(*m: int = 0*, *i: float = -1*, *d: float = -1*, *i_m: float = -1*, *d_m: float = -1*) → float

Convert to or from m'thly interest rates

Parameters

- **m** – m'thly frequency
- **i** – an annual-pay interest rate, to convert to m'thly
- **d** – or annual-pay discount rate, to convert to m'thly

- **i_m** – or m'thly interest rate, to convert to annual pay
- **d_m** – or m'thly discount rate, to convert to annual pay

Examples

```
>>> i = Interest.mthly(i_m=0.05, m=12)
>>> print("Convert mthly to annual-pay:", i)
>>> print("Convert annual-pay to mthly:", Interest.mthly(i=i, m=12))
```

static double_force(*i: float = -1.0*, *delta: float = -1.0*, *d: float = -1.0*, *v: float = -1.0*)

Double the force of interest

Parameters

- **i** – interest rate to double force of interest
- **d** – or discount rate
- **v** – or discount factor
- **delta** – or continuous rate

Returns

interest rate, of same form as input rate, after doubling force of interest

Examples

```
>>> print("Double force of interest of i =", Interest.double_force(i=0.05))
>>> print("Double force of interest of d =", Interest.double_force(d=0.05))
```

2.3 life

Life Contingent Risks - Applies probability laws

MIT License. Copyright (c) 2022-2023 Terence Lim

class actuarialmath.life.**Life**(***kwargs*)

Bases: *Actuarial*

Compute moments and probabilities

set_interest(***interest*) → *Life*

Set interest rate, which can be given in any form

Parameters

- **i** – assumed annual interest rate
- **d** – or assumed discount rate
- **v** – or assumed discount factor
- **delta** – or assumed contiuously compounded interest rate
- **v_t** – or assumed discount rate as a function of time
- **i_m** – or assumed monthly interest rate

- **d_m** – or assumed monthly discount rate
- **m** – m'thly frequency, if i_m or d_m are given

static variance(*a*, *b*, *var_a*, *var_b*, *cov_ab: float*) → float

Variance of weighted sum of two r.v.

Parameters

- **a** – weight on first r.v.
- **b** – weight on other r.v.
- **var_a** – variance of first r.v.
- **var_b** – variance of other r.v.
- **cov_ab** – covariance of the r.v.'s

static covariance(*a*, *b*, *ab: float*) → float

Covariance of two r.v.

Parameters

- **a** – expected value of first r.v.
- **b** – expected value of other r.v.
- **ab** – expected value of product of the two r.v.

static bernoulli(*p*, *a: float = 1*, *b: float = 0*, *variance: bool = False*) → float

Mean or variance of bernoulli r.v. with values {a, b}

Parameters

- **p** – probability of first value
- **a** – first value
- **b** – other value
- **variance** – whether to return variance (True) or mean (False)

static binomial(*p: float*, *N: int*, *variance: bool = False*) → float

Mean or variance of binomial r.v.

Parameters

- **p** – probability of occurence
- **N** – number of trials
- **variance** – whether to return variance (True) or mean (False)

static mixture(*p*, *p1*, *p2: float*, *N: int = 1*, *variance: bool = False*) → float

Mean or variance of binomial mixture

Parameters

- **p** – probability of selecting first r.v.
- **p1** – probability of occurrence if first r.v.
- **p2** – probability of occurrence if other r.v.
- **N** – number of trials
- **variance** – whether to return variance (True) or mean (False)

Examples

```
>>> p1 = (1. - 0.02) * (1. - 0.01)  # 2_p_x if vaccine given
>>> p2 = (1. - 0.02) * (1. - 0.02)  # 2_p_x if vaccine not given
>>> math.sqrt(Life.mixture(p=.2, p1=p1, p2=p2, N=100000, variance=True))
```

static conditional_variance(*p*, *p1*, *p2: float*, *N: int = 1*) → float

Conditional variance formula for mixture of binomials

Parameters

- **p** – probability of selecting first r.v.
- **p1** – probability of occurence for first r.v.
- **p2** – probability of occurence for other r.v.
- **N** – number of trials

Examples

```
>>> p1 = (1. - 0.02) * (1. - 0.01)  # 2_p_x if vaccine given
>>> p2 = (1. - 0.02) * (1. - 0.02)  # 2_p_x if vaccine not given
>>> math.sqrt(Life.mixture(p=.2, p1=p1, p2=p2, N=100000, variance=True))
```

static portfolio_percentile(*mean: float*, *variance: float*, *prob: float*, *N: int = 1*) → float

Probability percentile of the sum of N iid r.v.'s

Parameters

- **mean** – mean of each independent obsevation
- **variance** – variance of each independent observation
- **prob** – probability threshold
- **N** – number of observations to sum

static portfolio_cdf(*mean: float*, *variance: float*, *value: float*, *N: int = 1*) → float

Probability distribution of a value in the sum of N iid r.v.

Parameters

- **mean** – mean of each independent obsevation
- **variance** – variance of each independent observation
- **value** – value to compute probability distribution in the sum
- **N** – number of observations to sum

static quantiles_frame(*quantiles: List[float] = [0.8, 0.85, 0.9, 0.95, 0.975, 0.99, 0.995]*) → Any

Display selected quantile values from Normal distribution table

Parameters

quantiles – list of quantiles to display normal distribution values

2.4 survival

Survival models - Computes survival and mortality functions

class actuarialmath.survival.**Survival**(***kwargs*)

Bases: *Life*

Set and derive basic survival and mortality functions

set_survival(*S: Callable[[int, float, float], float] | None = None*, *f: Callable[[int, float, float], float] | None = None*, *l: Callable[[int, float], float] | None = None*, *mu: Callable[[int, float], float] | None = None*, *minage: int = 0*, *maxage: int = 1000*) → *Survival*

Construct the basic survival and mortality functions given any one form

Parameters

- **S** – probability [x]+s survives t years
- **f** – or lifetime density function of [x]+s after t years
- **l** – or number of lives aged (x+t)
- **mu** – or force of mortality at age (x+t)
- **maxage** – maximum age
- **minage** – minimum age

Examples:

```
>>> B, c = 0.00027, 1.1
>>> def S(x,s,t): return (math.exp(-B * c**(x+s) * (c**t - 1)/math.log(c)))
>>> life = Survival().set_survival(S=S)
```

```
>>> def ell(x,s): return (1 - (x+s) / 60)**(1 / 3)
>>> life = Survival().set_survival(l=ell)
```

l_x(*x: int*, *s: int = 0*) → float

Number of lives at integer age [x]+s: l_[x]+s

Parameters

- **x** – age of selection
- **s** – years after selection

d_x(*x: int*, *s: int = 0*) → float

Number of deaths at integer age [x]+s: d_[x]+s

Parameters

- **x** – age of selection
- **s** – years after selection

p_x(*x: int*, *s: int = 0*, *t: int = 1*) → float

Probability that [x]+s lives another t years: : t_p_[x]+s

Parameters

- **x** – age of selection

- **s** – years after selection
- **t** – survives at least t years

q_x(*x: int*, *s: int = 0*, *t: int = 1*, *u: int = 0*) → float

Probability that [x]+s lives for u, but not t+u years: u|t_q_[x]+s

Parameters

- **x** – age of selection
- **s** – years after selection
- **u** – survives u years, then
- **t** – dies within next t years

Examples:

```
>>> def ell(x,s): return (1-((x+s)/250)) if x+s < 40 else (1-((x+s)/100)**2)
>>> q = Survival().set_survival(l=ell).q_x(30, t=20)
```

f_x(*x: int*, *s: int = 0*, *t: int = 0*) → float

Lifetime density function of [x]+s after t years: f_[x]+s(t)

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – dies at year t

Examples:

```
>>> B, c = 0.00027, 1.1
>>> def S(x,s,t): return (math.exp(-B * c**(x+s) * (c**t - 1)/math.log(c)))
>>> f = Survival().set_survival(S=S).f_x(x=50, t=10)
```

mu_x(*x: int*, *s: int = 0*, *t: int = 0*) → float

Force of mortality of [x] at s+t years: mu_[x](s+t)

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – force of mortality at year t

Examples

```
>>> def ell(x, s): return (1 - (x+s) / 60)**(1 / 3)
>>> print(Survival().set_survival(l=ell).mu_x(35))
```

2.5 lifetime

Future lifetimes - Computes expectations and moments of future lifetime

class actuarialmath.lifetime.**Lifetime**(***kwargs*)

Bases: *Survival*

Computes expected moments of future lifetime

e_x(*x: int*, *s: int = 0*, *t: int = -999*, *curtate: bool = True*, *moment: int = 1*) → float

Compute curtate or complete expectations and moments of life

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – limited at t years
- **curtate** – whether curtate (True) or complete (False) expectations
- **moment** – whether to compute first (1) or second (2) moment

Examples

```
>>> def l(x, s): return 0. if (x+s) >= 100 else 1 - ((x + s)**2) / 10000.
>>> print(Lifetime().set_survival(l=l).e_x(75, t=10, curtate=False))
```

2.6 fractional

Fractional age assumptions- Computes survival and mortality functions between integer ages.

class actuarialmath.fractional.**Fractional**(*udd: bool = True*, ***kwargs*)

Bases: *Lifetime*

Compute survival functions at fractional ages and durations

Parameters
udd – select UDD (True, default) or CFM (False) between integer ages

l_r(*x: int*, *s: int = 0*, *r: float = 0.0*) → float

Number of lives at fractional age: l_[x]+s+r

Parameters

- **x** – age of selection
- **s** – years after selection
- **r** – fractional year after selection

p_r(*x: int*, *s: int = 0*, *r: float = 0.0*, *t: float = 1.0*) → float

Probability of survival from and through fractional age: t_p_[x]+s+r

Parameters

- **x** – age of selection
- **s** – years after selection
- **r** – fractional year after selection
- **t** – fractional number of years survived

Examples:

```
>>> life = Fractional(udd=False).set_survival(l=lambda x,t: 50-x-t)
>>> print(life.p_r(47, r=0.), life.p_r(47, r=0.5), life.p_r(47, r=1.))
```

q_r(*x: int*, *s: int = 0*, *r: float = 0.0*, *t: float = 1.0*, *u: float = 0.0*) → float

Deferred mortality rate within fractional ages: u|t_q_[x]+s+r

Parameters

- **x** – age of selection
- **s** – years after selection
- **r** – fractional year after selection
- **u** – fractional number of years survived, then
- **t** – death within next fractional years t

Examples

```
>>> life = Fractional(udd=False).set_survival(l=lambda x,t: 50-x-t)
>>> print(life.q_r(x, r=0.5))
```

mu_r(*x: int*, *s: int = 0*, *r: float = 0.0*) → float

Force of mortality at fractional age: mu_[x]+s+r

Parameters

- **x** – age of selection
- **s** – years after selection
- **r** – fractional year after selection

f_r(*x: int*, *s: int = 0*, *r: float = 0.0*, *t: float = 0.0*) → float

Lifetime density function at fractional age: f_[x]+s+r (t)

Parameters

- **x** – age of selection
- **s** – years after selection
- **r** – fractional year after selection
- **t** – death at fractional year t

E_r(*x: int*, *s: int = 0*, *r: float = 0.0*, *t: float = 1.0*) → float

Pure endowment at fractional age: t_E_[x]+s+r

Parameters

- **x** – age of selection
- **s** – years after selection
- **r** – fractional year after selection
- **t** – limited at fractional year t

e_r(*x: int*, *s: int = 0*, *t: float = -999*) → float

Temporary expected future lifetime at fractional age: e_[x]+s:t

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – fractional year limit of expected future lifetime

static e_approximate(*e_complete: float = None*, *e_curtate: float = None*, *variance: bool = False*) → float

Convert between curtate and complete expectations assuming UDD shortcut

Parameters

- **e_complete** – complete expected lifetime
- **e_curtate** – or curtate expected lifetime
- **variance** – to approximate mean (False) or variance (True)

Returns

approximate complete or curtate expectation assuming UDD

Examples

```
>>> print(Fractional.e_approximate(e_complete=15))
>>> print(Fractional.e_approximate(e_curtate=15))
```

2.7 insurance

Life insurance - Computes present values of insurance benefits

class actuarialmath.insurance.**Insurance**(*udd: bool = True*, ***kwargs*)

Bases: *Fractional*

Compute expected present values of life insurance

E_x(*x: int*, *s: int = 0*, *t: int = 1*, *endowment: int = 1*, *moment: int = 1*) → float

Pure endowment: t_E_x

Parameters

- **x** – age of selection

- **s** – years after selection
- **t** – term of pure endowment
- **endowment** – amount of pure endowment
- **moment** – compute first or second moment

Examples:

```
>>> life = Insurance().set_survival(mu=lambda x,t: .02*t).set_interest(i=.03)
>>> var = life.E_x(0, t=15, moment=life.VARIANCE, endowment=10000)
```

A_x(*x: int*, *s: int = 0*, *t: int = -999*, *u: int = 0*, *benefit: ~typing.Callable = <function Insurance.<lambda>>*, *endowment: float = 0.0*, *moment: int = 1*, *discrete: bool = True*) → float

Numerically compute EPV of insurance from basic survival functions

Parameters

- **x** – age of selection
- **s** – years after selection
- **u** – year deferred
- **t** – term of insurance
- **benefit** – benefit as a function of age and year
- **endowment** – amount of endowment for endowment insurance
- **moment** – compute first or second moment
- **discrete** – benefit paid yearend (True) or moment of death (False)

Examples:

```
>>> life = Insurance().set_interest(delta=.05)        >>>                    .
↪set_survival(mu=lambda x,s: .03)
>>> benefit = lambda x,t: math.exp(0.04 * t)
>>> A = life.A_x(0, benefit=benefit)
>>> print(A)    # 0.75
>>> A2 = life.A_x(0, moment=2, benefit=benefit)
>>> print(A2)  #0.60
```

static insurance_variance(*A2: float*, *A1: float*, *b: float = 1*) → float

Compute variance of insurance given moments and benefit

Parameters

- **A2** – second moment of insurance r.v.
- **A1** – first moment of insurance r.v.
- **b** – benefit amount

whole_life_insurance(*x: int*, *s: int = 0*, *moment: int = 1*, *b: int = 1*, *discrete: bool = True*) → float

Whole life insurance: A_x

Parameters

- **x** – age of selection
- **s** – years after selection

- **b** – amount of benefit
- **moment** – compute first or second moment
- **discrete** – benefit paid year-end (True) or moment of death (False)

Examples:

```
>>> life.whole_life_insurance(x=0)
```

term_insurance(*x: int*, *s: int = 0*, *t: int = 1*, *b: int = 1*, *moment: int = 1*, *discrete: bool = True*) → float

Term life insurance: A_x:t^1

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – term of insurance
- **b** – amount of benefit
- **moment** – compute first or second moment
- **discrete** – benefit paid year-end (True) or moment of death (False)

Examples:

```
>>> life.term_insurance(x=0, t=30)
```

deferred_insurance(*x: int*, *s: int = 0*, *u: int = 0*, *t: int = -999*, *b: int = 1*, *moment: int = 1*, *discrete: bool = True*) → float

Deferred insurance n|_A_x:t^1 = discounted term or whole life

Parameters

- **x** – age of selection
- **s** – years after selection
- **u** – year deferred
- **t** – term of insurance
- **b** – amount of benefit
- **moment** – compute first or second moment
- **discrete** – benefit paid year-end (True) or moment of death (False)

Examples:

```
>>> life.deferred_insurance(x=0, u=10, t=20)
```

endowment_insurance(*x: int*, *s: int = 0*, *t: int = 1*, *b: int = 1*, *endowment: int = -1*, *moment: int = 1*, *discrete: bool = True*) → float

Endowment insurance: A_x^1:t = term insurance + pure endowment

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – term of insurance

- **b** – amount of benefit
- **endowment** – amount of endowment paid at end of term if survive
- **moment** – compute first or second moment
- **discrete** – benefit paid year-end (True) or moment of death (False)

Examples:

```
>>> life.endowment_insurance(x=0, t=10)
```

increasing_insurance(*x: int*, *s: int = 0*, *t: int = -999*, *b: int = 1*, *discrete: bool = True*) → float

Increasing life insurance: (IA)_x

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – term of insurance
- **b** – amount of benefit in first year
- **discrete** – benefit paid year-end (True) or moment of death (False)

Examples:

```
>>> life.increasing_insurance(x=0, t=10)
```

decreasing_insurance(*x*, *s: int = 0*, *t: int = 1*, *b: int = 1*, *discrete: bool = True*) → float

Decreasing life insurance: (DA)_x

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – term of insurance
- **b** – amount of benefit in first year
- **discrete** – benefit paid year-end (True) or moment of death (False)

Examples:

```
>>> life.decreasing_insurance(x=0, t=10)
```

Z_x(*x*, *s: int = 0*, *t: float = 0.0*, *discrete: bool = True*)

EPV of year t insurance death benefit for life aged [x]+s: b_x[s]+s(t)

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – year of benefit
- **discrete** – benefit paid year-end (True) or moment of death (False)

Z_t(*x: int*, *prob: float*, *discrete: bool = True*) → float

T_x, given the prob of the PV of life insurance, i.e. r.v. Z(t)

Parameters

- **x** – age initially insured
- **prob** – desired probability threshold
- **discrete** – benefit paid year-end (True) or moment of death (False)

Returns

T s.t. S(t)==prob; if discrete, return K=floor(T) s.t. S(K)>=prob

Examples

```
>>> t = life.Z_t(x=20, prob=0.8, discrete=True)
```

Z_from_t(*t: float*, *discrete: bool = True*) → float

PV of annual or continuous insurance payment Z(t) at t=T_x

Parameters

- **t** – year of death
- **b** – benefit paid at t
- **discrete** – benefit paid year-end (True) or moment of death (False)

Examples

```
>>> t = life.Z_t(x=20, prob=0.8, discrete=True)
>>> print(Z, life.Z_from_t(t, discrete=True))
```

Z_from_prob(*x: int*, *prob: float*, *discrete: bool = True*) → float

Percentile of annual or continuous WL insurance PV r.v. Z given probability

Parameters

- **x** – age initially insured
- **prob** – threshold for probability of survival
- **discrete** – benefit paid year-end (True) or moment of death (False)

Examples

```
>>> Z = life.Z_from_prob(x=20, prob=0.8, discrete=True)
```

Z_to_t(*Z: float*) → float

T_x s.t. PV of continuous WL insurance payment is Z

Parameters

Z – Present value of benefit paid

Examples

```
>>> t = life.Z_t(x=20, prob=0.8, discrete=False)
>>> Z = life.Z_from_prob(x=20, prob=0.8, discrete=False)
>>> print(t, life.Z_to_t(Z))
```

Z_to_prob(*x: int*, *Z: float*) → float

Probability that continuous WL insurance PV r.v. is no more than Z

Parameters

- **x** – age initially insured
- **Z** – present value of benefit paid

Examples

```
>>> Z = life.Z_from_prob(x=20, prob=0.8, discrete=False)
>>> print(life.Z_to_prob(x=20, Z=Z))
```

Z_plot(*x: int*, *s: int = 0*, *stop: int = 0*, *benefit: ~typing.Callable = <function Insurance.<lambda>>*, *T: float | None = None*, *discrete: bool = True*, *ax: ~typing.Any = None*, *dual: bool = False*, *title: str | None = None*, *color: str = 'r'*, *alpha: float = 0.3*) → float | None

Plot of PV of insurance r.v. Z vs t

Parameters

- **x** – age of selection
- **s** – years after selection
- **stop** – time to end plot
- **benefit** – benefit as a function of age and time
- **T** – specific time to annotate
- **insurance** (*discrete discrete or continuous*) –
- **ax** – figure object to plot in
- **dual** – whether to plot survival function on secondary axis
- **color** – color of primary plot
- **alpha** – transparency of plot area
- **title** – title of plot

2.8 annuity

Life annuities - Computes present values of annuity benefits

MIT License. Copyright (c) 2022-2023 Terence Lim

class actuarialmath.annuity.**Annuity**(*udd: bool = True*, ***kwargs*)

Bases: *Insurance*

Compute present values and relationships of life annuities

a_x(*x: int*, *s: int = 0*, *t: int = -999*, *u: int = 0*, *benefit: ~typing.Callable = <function Annuity.<lambda>>*, *discrete: bool = True*) → float

Compute EPV of life annuity from survival function

Parameters

- **x** – age of selection
- **s** – years after selection
- **u** – year deferred
- **t** – term of insurance
- **benefit** – benefit as a function of age and year
- **discrete** – annuity due (True) or continuous (False)

Examples

```
>>> life.a_x(x=20)
```

immediate_annuity(*x: int*, *s: int = 0*, *t: int = -999*, *b: int = 1*, *variance=False*) → float

Compute EPV of immediate life annuity

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – term of insurance
- **b** – benefit amount
- **variance** – return EPV (False) or variance (True)

Examples

```
>>> life.immediate_annuity(x=20, t=30)
```

annuity_twin(*A: float*, *discrete: bool = True*) → float

Returns annuity from its WL or Endowment Insurance twin"

Parameters

- **A** – cost of insurance
- **discrete** – discrete/annuity due (True) or continuous (False)

Examples

```
>>> life.annuity_twin(A=0.05879)
```

insurance_twin(*a: float*, *moment: int = 1*, *discrete: bool = True*) → float

Returns WL or Endowment Insurance twin from annuity

Parameters

- **a** – cost of annuity
- **discrete** – discrete/annuity due (True) or continuous (False)

Examples

```
>>> life.insurance_twin(a=19.7655)
```

annuity_variance(*A2: float*, *A1: float*, *b: float = 1.0*, *discrete: bool = True*) → float

Compute variance from WL or endowment insurance twin

Parameters

- **A2** – second moment of insurance factor
- **A1** – first moment of insurance factor
- **b** – annuity benefit amount
- **discrete** – discrete/annuity due (True) or continuous (False)

Examples

```
>>> life.annuity_variance(A2=0.22, A1=0.45)
```

whole_life_annuity(*x: int*, *s: int = 0*, *b: int = 1*, *variance: bool = False*, *discrete: bool = True*) → float

Whole life annuity: a_x

Parameters

- **x** – age of selection
- **s** – years after selection
- **b** – annuity benefit amount
- **variance** – return EPV (True) or variance (False)
- **discrete** – annuity due (True) or continuous (False)

Examples

```
>>> life.whole_life_annuity(x=24, variance=True, due=False)
```

temporary_annuity(*x: int*, *s: int = 0*, *t: int = -999*, *b: int = 1*, *variance: bool = False*, *discrete: bool = True*) → float

Temporary life annuity: a_x:t

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – term of annuity in years
- **b** (*int*) – annuity benefit amount
- **variance** – return EPV (True) or variance (False)
- **discrete** – annuity due (True) or continuous (False)

Examples

```
>>> life.temporary_annuity(x=24, t=10)
```

deferred_annuity(*x: int*, *s: int = 0*, *u: int = 0*, *t: int = -999*, *b: int = 1*, *discrete: bool = True*) → float

Deferred life annuity n|t_a_x = n+t_a_x - n_a_x

Parameters

- **x** – age of selection
- **s** – years after selection
- **u** – years deferred
- **t** – term of annuity in years
- **b** – annuity benefit amount
- **discrete** – annuity due (True) or continuous (False)

Examples

```
>>> life.deferred_annuity(x=24, u=5, t=10, discrete=False)
```

certain_life_annuity(*x: int*, *s: int = 0*, *u: int = 0*, *t: int = -999*, *b: int = 1*, *discrete: bool = True*) → float

Certain and life annuity = certain + deferred

Parameters

- **x** – age of selection
- **s** – years after selection
- **u** – years of certain annuity
- **t** – term of life annuity in years
- **b** – annuity benefit amount

- **discrete** – annuity due (True) or continuous (False)

Examples

```
>>> life.certain_life_annuity(x=24, u=5, t=10)
```

increasing_annuity(*x: int*, *s: int = 0*, *t: int = -999*, *b: int = 1*, *discrete: bool = True*) → float

Increasing annuity

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – term of life annuity in years
- **b** – benefit amount at end of first year
- **discrete** – annuity due (True) or continuous (False)

Examples

```
>>> life.increasing_annuity(x=24, t=10)
```

decreasing_annuity(*x: int*, *s: int = 0*, *t: int = 0*, *b: int = 1*, *discrete: bool = True*) → float

Identity (Da)_x:n + (Ia)_x:n = (n+1) a_x:n temporary annuity

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – term of life annuity in years
- **b** – benefit amount at end of first year
- **discrete** – annuity due (True) or continuous (False)

Examples

```
>>> life.decreasing_annuity(x=24, t=10)
```

Y_t(*x: int*, *prob: float*, *discrete: bool = True*) → float

T_x given percentile of the r.v. Y = PV of WL or Temporary Annuity

Parameters

- **x** – age of selection
- **prob** – desired probability threshold
- **discrete** – annuity due (True) or continuous (False)

Returns

T s.t. S(T) == 1-prob; if discrete, return K=ceil(T) s.t. S(K) <= 1-prob

Y_x(*x*, *s: int = 0*, *t: float = 1.0*, *discrete: bool = True*) → float

EPV of year t annuity benefit for life aged [x]+s: b_x[s]+s(t)

Parameters

- **x** – age initially insured
- **s** – years after selection
- **t** – year of benefit
- **discrete** – annuity due (True) or continuous (False)

Y_from_t(*t: float*, *discrete: bool = True*) → float

PV of insurance payment Y(t), given T_x

Parameters

- **t** – year of death
- **discrete** – annuity due (True) or continuous (False)

Y_from_prob(*x: int*, *prob: float*, *discrete: bool = True*) → float

Percentile of annual or continuous WL annuity PV r.v. Y, given probability

Parameters

- **x** – age initially insured
- **prob** – desired probability threshold
- **discrete** – annuity due (True) or continuous (False)

Y_to_t(*Y: float*) → float

T_x s.t. PV of continuous WL annuity payments is Y

Parameters
Y – Present value of benefits paid

Y_to_prob(*x: int*, *Y: float*) → float

Probability that continuous WL annuity PV r.v. is no more than Y

Parameters

- **x** – age initially insured
- **Y** – present value of benefits paid

Y_plot(*x: int*, *s: int = 0*, *stop: int = 0*, *benefit: ~typing.Callable = <function Annuity.<lambda>>*, *T: float | None = None*, *discrete: bool = True*, *ax: ~typing.Any = None*, *dual: bool = False*, *title: str | None = None*, *color: str = 'r'*, *alpha: float = 0.3*) → float | None

Plot PV of annuity r.v. Y vs T

Parameters

- **x** – age of selection
- **s** – years after selection
- **stop** – time to end plot
- **benefit** – benefit as a function of selection age and time
- **discrete** – discrete or continuous insurance
- **ax** – figure object to plot in

- **dual** – whether to plot survival function on secondary axis
- **color** – color of primary plot
- **alpha** – transparency of plot area
- **title** – title of plot

2.9 premiums

Premiums - Computes net and gross premiums, with the equivalence principle

class actuarialmath.premiums.**Premiums**(*udd: bool = True*, ***kwargs*)

Bases: *Annuity*

Compute et and gross premiums under equivalence principle

net_premium(*x: int*, *s: int = 0*, *t: int = -999*, *u: int = 0*, *n: int = 0*, *b: int = 1*, *endowment: int = 0*, *discrete: bool | None = True*, *return_premium: bool = False*, *annuity: bool = False*, *initial_cost: float = 0.0*) → float

Net level premium for special n-pay, u-deferred t-year term insurance

Parameters

- **x** – age initially insured
- **s** – years after selection
- **u** – years of deferral
- **n** – number of years of premiums paid
- **t** – year of death
- **b** – benefit amount
- **endowment** – endowment amount
- **initial_cost** – EPV of any other expenses or benefits, if any
- **return_premium** – if premiums without interest refunded at death
- **annuity** – whether benefit is insurance (False) or annuity
- **discrete** – whether annuity due (True) or continuous (False)

Examples

```
>>> life = Premiums().set_interest(delta=0.06)          >>>                 .
↪set_survival(mu=lambda x,s: 0.04)
>>> life.net_premium(x=0)
```

insurance_equivalence(*premium: float*, *b: int = 1*, *discrete: bool = True*) → float

Compute whole life or endowment insurance factor, given net premium

Parameters

- **premium** – level net premium amount

- **b** – benefit amount
- **discrete** – discrete/annuity due (True) or continuous (False)

Returns
Insurance factor value, given net premium under equivalence principle

Examples

```
>>> life = Premiums().set_interest(d=0.05)
>>> life.insurance_equivalence(premium=2143, b=100000)
```

annuity_equivalence(*premium: float*, *b: int = 1*, *discrete: bool = True*) → float

Compute whole life or temporary annuity factor, given net premium

Parameters

- **premium** – level net premium amount
- **b** – benefit amount
- **discrete** – discrete/annuity due (True) or continuous (False)

Returns
Annuity factor value, given net premium under equivalence principle

Examples

```
>>> life = Premiums().set_interest(d=0.05)
>>> a = life.annuity_equivalence(premium=2143, b=100000)
```

premium_equivalence(*A: float | None = None*, *a: float | None = None*, *b: int = 1*, *discrete: bool = True*) → float

Compute premium from whole life or endowment insurance and annuity factors

Parameters

- **A** – insurance factor
- **a** – annuity factor
- **b** – insurance benefit amount
- **discrete** – annuity due (True) or continuous (False)

Returns
Net premium under equivalence principle

gross_premium(*a: float | None = None*, *A: float | None = None*, *IA: float = 0*, *discrete: bool = True*, *benefit: float = 1*, *E: float = 0*, *endowment: int = 0*, *settlement_policy: float = 0.0*, *initial_policy: float = 0.0*, *initial_premium: float = 0.0*, *renewal_policy: float = 0.0*, *renewal_premium: float = 0.0*) → float

Gross premium by equivalence principle

Parameters

- **A** – insurance factor
- **a** – annuity factor

- **IA** – increasing insurance factor, to return premiums w/o interest
- **E** – pure endowment factor for endowment benefit
- **benefit** – insurance benefit amount
- **endowment** – endowment benefit amount
- **settlement_policy** – settlement expense per policy
- **initial_policy** – initial expense per policy
- **renewal_policy** – renewal expense per policy
- **initial_premium** – initial premium per $ of gross premium
- **renewal_premium** – renewal premium per $ of gross premium
- **discrete** – annuity due (True) or continuous (False)

Examples

```
>>> Premiums().gross_premium(a=0.17094,
>>                          A=6.8865,
>>                          IA= 0.96728,
>>                          benefit=100000,
>>                          initial_premium=0.5,
>>                          renewal_premium=.05,
>>                          renewal_policy=200,
>>                          initial_policy=200)
```

2.10 policyvalues

Policy Values - Computes present value of future losses and reserves

class actuarialmath.policyvalues.**PolicyValues**(***kwargs*)

Bases: *Premiums*

Compute net and gross future losses and policy values

net_future_loss(*A: float*, *A1: float*, *b: int = 1*) → float

Shortcut for net policy value with WL or Endowment Insurance factors

Parameters

- **A** – insurance factor at age (x)
- **A1** – insurance factor at t years after x
- **b** – benefit amount

net_variance_loss(*A1: float*, *A2: float*, *A: float = 0*, *b: int = 1*) → float

Variance of net loss with WL or Endowment Insurance factors

Parameters

- **A** – insurance factor at age (x)
- **A1** – first moment of insurance factor at t years after x

- **A2** – insurance factor at double force of interest t years after x
- **b** – benefit amount

net_policy_variance(*x*, *s: int = 0*, *t: int = 0*, *b: int = 1*, *n: int = -999*, *endowment: int = 0*, *discrete: bool = True*) → float

Variance of net future loss for WL or Endowment Insurance only

Parameters

- **x** – age of selection
- **s** – years after selection
- **n** – term of life insurance
- **t** – years after issue to compute policy variance
- **b** – benefit amount
- **endowment** – endowment amount
- **discrete** – annuity due (True) or continuous (False)

net_policy_value(*x: int*, *s: int = 0*, *t: int = 0*, *b: int = 1*, *n: int = -999*, *endowment: int = 0*, *discrete: bool = True*) → float

Net policy value assuming net premiums from equivalence principle: E[L_t]

Parameters

- **x** – age initially insured
- **s** – years after selection
- **n** – term of life insurance
- **t** – years after issue to compute policy variance
- **b** – benefit amount
- **endowment** – endowment amount
- **discrete** – discrete/annuity due (True) or continuous (False)

gross_future_loss(*A: float | None = None*, *a: float | None = None*, *contract:* Contract *| None = None*) → float

Shortcut for gross policy value with WL or Endowment Insurance factors

Parameters

- **A** – insurance factor at age (x)
- **a** – annuity factor at age (x)
- **contract** – policy contract terms and expenses

gross_variance_loss(*A1: float*, *A2: float = 0*, *contract:* Contract *| None = None*) → float

Variance of gross loss with WL or endowment insurance factors

Parameters

- **A1** – insurance factor
- **A2** – insurance factor at double the force of interest
- **policy** – policy terms and expenses

gross_policy_variance(*x: int*, *s: int = 0*, *t: int = 0*, *n: int = -999*, *contract:* Contract | *None = None*) → float

Variance of gross policy value for WL and Endowment Insurance only

Parameters

- **x** – age initially insured
- **s** – years after selection
- **n** – term of life insurance
- **t** – years after issue to compute policy variance
- **contract** – policy contract terms and expenses

gross_policy_value(*x: int*, *s: int = 0*, *t: int = 0*, *n: int = -999*, *contract:* Contract | *None = None*) → float

Gross policy values for insurance: t_V = E[L_t]

Parameters

- **x** – age initially insured
- **s** – years after selection
- **t** – number of years of premiums paid
- **n** – term of insurance
- **contract** – policy contract terms

L_from_t(*t: float*, *contract:* Contract | *None = None*) → float

PV of Loss L(t) at time of death t = T_x

Parameters

- **t** – year of death
- **contract** – policy contract

L_from_prob(*x: int*, *prob: float*, *contract:* Contract | *None = None*) → float

Percentile of PV future loss r.v. L given probability

Parameters

- **x** – age
- **prob** – probability threshold
- **contract** – policy contract

L_to_t(*L: float*, *contract:* Contract | *None = None*) → float

Time of death T_x s.t. PV future loss is no more than L

Parameters

- **L** – PV of future loss
- **contract** – policy contract terms and expenses

L_to_prob(*x: int*, *L: float*, *contract: ~actuarialmath.policyvalues.Contract = <actuarialmath.policyvalues.Contract object>*) → float

Probability such that PV of future loss r.v. is no more than L”

Parameters

- **x** – age selected

- **L** – PV of future loss
- **contract** – policy contract terms and expenses

L_plot(*x: int*, *s: int = 0*, *stop: int = 0*, *T: float | None = None*, *contract:* Contract *| None = None*, *ax: Any = None*, *dual: bool = False*, *title: str | None = None*, *color: str = 'r'*, *alpha: float = 0.3*) → float | None

Plot PV of future loss r.v. L vs time of death T_x

Parameters

- **x** – age selected
- **s** – years after selection
- **stop** – time to end plot
- **contract** – policy contract terms and expenses
- **T** – point in time to indicate probability and loss values
- **ax** – figure object to plot in
- **title** – title of plot
- **dual** – whether to plot survival function on secondary axis
- **color** – color to plot curve
- **alpha** – transparency of plot area

class actuarialmath.policyvalues.**Contract**(*premium: float = 1*, *initial_policy: float = 0*, *initial_premium: float = 0*, *renewal_policy: float = 0*, *renewal_premium: float = 0*, *settlement_policy: float = 0*, *benefit: float = 1*, *endowment: float = 0*, *T: int = -999*, *discrete: bool = True*)

Bases: *Actuarial*

Set and retrieve policy contract terms

Parameters

- **premium** – level premium amount
- **benefit** – insurance death benefit amount
- **settlement_policy** – settlement expense per policy
- **endowment** – endowment benefit amount
- **initial_policy** – first year total expense per policy
- **initial_premium** – first year total premium per $ of gross premium
- **renewal_policy** – renewal expense per policy
- **renewal_premium** – renewal premium per $ of gross premium
- **discrete** – annuity due (True) or continuous (False)
- **T** – term of insurance
- **discrete** – annuity due (True) or continuous (False)

set_contract(***terms*) → Any

Update any existing policy contract terms

Parameters

****kwargs** – one or more contract policy terms, and its value to update

property premium_terms: Dict

Dict of terms required for calculating gross premiums

Examples

```
>>> life = PolyValues().set_interest(i=0.04)
>>> contract = Contract(premium=2.338, benefit=100, initial_premium=.1,
>>>                     renewal_premium=0.05)
>>> contract.premium = life.gross_premium(a=12, A=life.insurance_twin(a),
>>>                                       **contract.premium_terms)
```

renewals(*t: int = 0*) → *Contract*

Returns contract object with initial terms set to renewal terms

Parameters

t – number of years after initial

property renewal_profit: float

Renewal dollar profit (premium less renewal expenses)

property initial_cost: float

Total initial cost (net of renewal component of expenses and premiums)

property claims_cost: float

Total claims cost (death benefit + settlement expense)

2.11 reserves

Reserves - Computes recursive, interim or modified reserves

MIT License. Copyright 2022-2023 Terence Lim

class actuarialmath.reserves.**Reserves**(***kwargs*)

Bases: *PolyValues*

Compute recursive, interim or modified reserves

set_reserves(*T: int = 0*, *endowment: int | float = 0*, *V: Dict[int, float] | None = None*) → *Reserves*

Set values of the reserves table and the endowment benefit amount

Parameters

- **T** – max term of policy
- **V** – reserve values, keyed by time t
- **endowment** – endowment benefit amount

Examples

```
>>> life = Reserves().set_reserves(T=3)
```

fill_reserves(*x: int*, *s: int = 0*, *reserve_benefit: bool = False*, *contract: Contract | None = None*) → *Reserves*

Iteratively fill in missing values in reserves table

Parameters

- **x** – age selected
- **s** – starting from s years after selection
- **reserve_benefit** – whether benefit includes value of reserves
- **contract** – policy contract terms and expenses

V_plot(*ax: Any = None*, *color: str = 'r'*, *title: str = ''*)

Plot values from reserves tables

Parameters

- **title** – title to display
- **color** – color to plot curve

Examples

```
>>> life.V_plot(title=f"Reserves for term insurance")
```

reserves_frame()

Returns reserves table as a DataFrame

t_V_backward(*x: int*, *s: int = 0*, *t: int = 0*, *premium: float = 0*, *benefit: ~typing.Callable = <function Reserves.<lambda>>*, *per_premium: float = 0*, *per_policy: float = 0*, *reserve_benefit: bool = False*) → float | None

Backward recursion (with optional reserve benefit)

Parameters

- **x** – age selected
- **s** – starting s years after selection
- **t** – year of reserve to solve
- **benefit** – benefit amount at t+1
- **premium** – amount of premium paid just after t
- **per_premium** – expense per $ premium
- **per_policy** – expense per policy
- **reserve_benefit** – whether reserve value at t+1 included in benefit

t_V_forward(*x: int*, *s: int = 0*, *t: int = 0*, *premium: float = 0*, *benefit: ~typing.Callable = <function Reserves.<lambda>>*, *per_premium: float = 0*, *per_policy: float = 0*, *reserve_benefit: bool = False*) → float | None

Forward recursion (with optional reserve benefit)

Parameters

- **x** – age selected
- **s** – starting s years after selection
- **t** – year of reserve to solve
- **benefit** – benefit amount at t
- **premium** – amount of premium paid just after t-1
- **per_premium** – expense per $ premium
- **per_policy** – expense per policy
- **reserve_benefit** – whether reserve value at t included in benefit

t_V(*x: int*, *s: int = 0*, *t: int = 0*, *premium: float = 0*, *benefit: ~typing.Callable = <function Reserves.<lambda>>*, *reserve_benefit: bool = False*, *per_premium: float = 0*, *per_policy: float = 0*) → float | None

Solve year-t reserves by forward or backward recursion

Parameters

- **x** – age selected
- **s** – starting s years after selection
- **t** – year of reserve to solve
- **benefit** – benefit amount
- **premium** – amount of premium
- **per_premium** – expense per $ premium
- **per_policy** – expense per policy
- **reserve_benefit** – whether reserve value included in benefit

Examples

```
>>> G, x = 368.05, 0
>>> def fun(P):  # solve net premium from expense reserve equation
>>>     return life.t_V(x=x, t=2, premium=G-P, benefit=lambda t: 0,
>>>                     per_policy=5+.08*G)
>>> P = life.solve(fun, target=-23.64, grid=[.29, .31]) / 1000
```

r_V_backward(*x: int*, *s: int = 0*, *r: float = 0*, *benefit: int = 1*) → float | None

Backward recursion for interim reserves

Parameters

- **x** – age of selection
- **s** – years after selection
- **r** – solve for interim reserve at fractional year x+s+r
- **benefit** – benefit amount in year x+s+1

r_V_forward(*x: int*, *s: int = 0*, *r: float = 0*, *premium: float = 0*, *benefit: int = 1*) → float | None

Forward recursion for interim reserves

Parameters

- **x** – age of selection
- **s** – years after selection
- **r** – solve for interim reserve at fractional year x+s+r
- **benefit** – benefit amount in year x+s+1
- **premium** – premium amount just after year x+s

FPT_premium(*x: int*, *s: int = 0*, *n: int = -999*, *b: int = 1*, *first: bool = False*) → float

Initial or renewal Full Preliminary Term premiums

Parameters

- **x** – age of selection
- **s** – years after selection
- **n** – term of insurance
- **b** – benefit amount in year x+s+1
- **first** – calculate year 1 (True) or year 2+ (False) FPT premium

FPT_policy_value(*x: int*, *s: int = 0*, *t: int = 0*, *b: int = 1*, *n: int = -999*, *endowment: int = 0*, *discrete: bool = True*) → float

Compute Full Preliminary Term policy value at time t

Parameters

- **x** – age of selection
- **s** – years after selection
- **n** – term of insurance
- **t** – year of policy value to calculate
- **b** – benefit amount in year x+s+1
- **endowment** – endowment amount
- **discrete** – fully discrete (True) or continuous (False) insurance

2.12 recursion

Recursion - Applies recursion, shortcut and actuarial formulas

class actuarialmath.recursion.**PPrint**(*label: str*, **args*, ***kwargs*)

Bases: _Blog

Helper to display recursion steps as actuarial notation in latex format

static q(*x: int*, *s: int = 0*, *t: int = 1*, *u: int = 0*) → str

Return latex string representation of mortality u|t_q_[x+s] term

static p(*x: int*, *s: int = 0*, *t: int = 1*) → str

Return latex string representation of survival t_p_[x+s] term

static e(*x: int*, *s: int = 0*, *t: int = -999*, *curtate: bool = False*, *moment: int = 1*) → str

Return string representation of expected future lifetime e_[x+s]:t term

static E(*x: int*, *s: int = 0*, *t: int = 1*, *endowment: int = 1*, *moment: int = 1*) → str

Return latex string representation of pure endowment t_E_[x+s]:t term

static IA(*x: int*, *s: int = 0*, *t: int = -999*, *b: int = 1*, *discrete: bool = True*) → str

Return latex string representation of increasing insurance IA_[x+s]:t term

static DA(*x: int*, *s: int = 0*, *t: int = -999*, *b: int = 1*, *discrete: bool = True*) → str

Return latex string representation of decreasing insurance DA_[x+s]:t term

static A(*x: int*, *s: int = 0*, *t: int = -999*, *u: int = 0*, *b: int = 1*, *moment: int = 1*, *endowment: int = 0*, *discrete: bool = True*) → str

Return latex string representation of insurance u|_A_[x+s]:t term

static a(*x: int*, *s: int = 0*, *t: int = -999*, *u: int = 0*, *b: int = 1*, *variance: bool = False*, *discrete: bool = True*) → str

Return latex string representation of annuity u|_a_[x+s]:t term

static m(*moment: int*, ***kwargs*) → str

Return latex string representation of moment exponent

class actuarialmath.recursion.**Recursion**(*depth: int = 3*, *verbose: bool = True*, ***kwargs*)

Bases: *Reserves*

Solve by appling recursive, shortcut and actuarial formulas repeatedly

Parameters

- **depth** – maximum depth of recursions (default is 3)
- **verbose** – whether to echo recursion steps (True, default)

Notes

7 types of function values can be loaded for recursion computations:

- 'q' : (deferred) probability (x) dies in t years
- 'p' : probability (x) survives t years
- 'e' : (temporary) expected future lifetime, or moments
- 'A' : deferred, term, endowment or whole life insurance, or moments
- 'IA' : decreasing life insurance of t years
- 'DA' : increasing life insurance of t years
- 'a' : deferred, temporary or whole life annuity of t years, or moments

Blog(**args*, ***kwargs*)

Returns Blog instance to collect messages and display for this query

static blog_options(*latex: bool = False*, *notebook: bool = False*)

Static method to change display options for tracing the recursion steps

Parameters

- **latex** – display actuarial notation in latex (True) or raw text (False)
- **notebook** – display to jupyter or colab notebook (True) or terminal (False)

Notes

latex and notebook options are set to True if notebook is auto-detected

Examples:

```
>>> Recursion.blog_options(latex=False)                # display as raw text
>>> Recursion.blog_options(latex=True, notebook=True)  # display latex format
```

set_q(*val: float*, *x: int*, *s: int = 0*, *t: int = 1*, *u: int = 0*) → *Recursion*

Set mortality rate u|t_q_[x+s] to given value

Parameters

- **val** – value to set
- **x** – age of selection
- **s** – years after selection
- **u** – survive u years, then…
- **t** – death within next t years

Examples

```
>>> Recursion(depth=3).set_q(0.02, x=3)
```

q_x(*x: int*, *s: int = 0*, *t: int = 1*, *u: int = 0*) → float

Probability that [x]+s lives for u, but not t+u years: u|t_q_[x]+s

Parameters

- **x** – age of selection
- **s** – years after selection
- **u** – survives u years, then
- **t** – dies within next t years

Examples:

```
>>> def ell(x,s): return (1-((x+s)/250)) if x+s < 40 else (1-((x+s)/100)**2)
>>> q = Survival().set_survival(l=ell).q_x(30, t=20)
```

set_p(*val: float*, *x: int*, *s: int = 0*, *t: int = 1*) → *Recursion*

Set survival probability t_p_[x+s] to given value

Parameters

- **val** – value to set

- **x** – age of selection
- **s** – years after selection
- **t** – survives next t years

Examples

```
>>> Recursion(depth=3).set_p(0.99, x=0)
```

p_x(*x: int*, *s: int = 0*, *t: int = 1*) → float

Compute survival probability by calling recursion helper

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – survives at least t years

set_e(*val: float*, *x: int*, *s: int = 0*, *t: int = -999*, *curtate: bool = False*, *moment: int = 1*) → *Recursion*

Set expected future lifetime e_[x+s]:t to given value

Parameters

- **val** – value to set
- **x** – age of selection
- **s** – years after selection
- **t** – limit of expected future lifetime
- **curtate** – curtate (True) or complete expectation (False)
- **moment** – first or second moment of expected future lifetime

e_x(*x: int*, *s: int = 0*, *t: int = -999*, *curtate: bool = False*, *moment: int = 1*) → float

Compute expected future lifetime by calling recursion helper

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – limited at t years
- **curtate** – whether curtate (True) or complete (False) lifetime
- **moment** – whether to compute first (1) or second (2) moment

set_E(*val: float*, *x: int*, *s: int = 0*, *t: int = 1*, *endowment: int = 1*, *moment: int = 1*) → *Recursion*

Set pure endowment t_E_[x+s] to given value

Parameters

- **val** – value to set
- **x** – age of selection
- **s** – years after selection
- **t** – death within next t years

- **endowment** – endowment value
- **moment** – first or second moment of pure endowment

E_x(*x: int*, *s: int = 0*, *t: int = 1*, *endowment: int = 1*, *moment: int = 1*) → float

Compute pure endowment by calling recursion helper

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – term of pure endowment
- **endowment** – amount of pure endowment
- **moment** – compute first or second moment

set_IA(*val: float*, *x: int*, *s: int = 0*, *t: int = -999*, *b: int = 1*, *discrete: bool = True*) → *Recursion*

Set increasing insurance IA_[x+s]:t to given value

Parameters

- **val** – value to set
- **x** – age of selection
- **s** – years after selection
- **t** – term of increasing insurance
- **b** – benefit after year 1
- **discrete** – discrete or continuous increasing insurance

increasing_insurance(*x: int*, *s: int = 0*, *t: int = -999*, *b: int = 1*, *discrete: bool = True*) → float

Compute increasing insurance with recursive helper

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – term of insurance
- **b** – amount of benefit in first year
- **discrete** – benefit paid year-end (True) or moment of death (False)

set_DA(*val: float*, *x: int*, *s: int = 0*, *t: int = -999*, *b: int = 1*, *discrete: bool = True*) → *Recursion*

Set decreasing insurance DA_[x+s]:t to given value

Parameters

- **val** – value to set
- **x** – age of selection
- **s** – years after selection
- **t** – term of decreasing insurance
- **b** – benefit after year 1
- **discrete** – discrete or continuous decreasing insurance

decreasing_insurance(*x: int*, *s: int = 0*, *t: int = -999*, *b: int = 1*, *discrete: bool = True*) → float

Compute decreasing insurance by attempting recursive helper first

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – term of insurance
- **b** – amount of benefit in first year
- **discrete** – benefit paid year-end (True) or moment of death (False)

set_A(*val: float*, *x: int*, *s: int = 0*, *t: int = -999*, *u: int = 0*, *b: int = 1*, *moment: int = 1*, *endowment: int = 0*, *discrete: bool = True*) → *Recursion*

Set insurance u|_A_[x+s]:t to given value

Parameters

- **val** – value to set
- **x** – age of selection
- **s** – years after selection
- **u** – defer u years
- **t** – term of insurance
- **endowment** – endowment amount
- **discrete** – discrete (True) or continuous (False) insurance
- **moment** – first or second moment of insurance

Examples

```
>>> Recursion().set_interest(i=0.06).set_A(A, x=0)
```

whole_life_insurance(*x: int*, *s: int = 0*, *b: int = 1*, *discrete: bool = True*, *moment: int = 1*) → float

Compute whole life insurance A_x by attempting recursion and twin first

Parameters

- **x** – age of selection
- **s** – years after selection
- **b** – amount of benefit
- **moment** – compute first or second moment
- **discrete** – benefit paid year-end (True) or moment of death (False)

term_insurance(*x: int*, *s: int = 0*, *t: int = 1*, *b: int = 1*, *moment: int = 1*, *discrete: bool = True*) → float

Compute term life insurance A_x:t^1 by attempting recursion first

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – term of insurance

- **b** – amount of benefit
- **moment** – compute first or second moment
- **discrete** – benefit paid year-end (True) or moment of death (False)

deferred_insurance(*x: int*, *s: int = 0*, *b: int = 1*, *u: int = 0*, *t: int = -999*, *moment: int = 1*, *discrete: bool = True*) → float

Compute deferred life insurance u|A_x:t^1 by attempting recursion first

endowment_insurance(*x: int*, *s: int = 0*, *t: int = 1*, *b: int = 1*, *endowment: int = -1*, *moment: int = 1*, *discrete: bool = True*) → float

Compute endowment insurance u|A_x:t by attempting recursion first

set_a(*val: float*, *x: int*, *s: int = 0*, *t: int = -999*, *u: int = 0*, *b: int = 1*, *variance: bool = False*, *discrete: bool = True*) → *Recursion*

Set annuity u|_a_[x+s]:t to given value

Parameters

- **val** – value to set
- **x** – age of selection
- **s** – years after selection
- **u** – defer u years
- **t** – term of annuity
- **b** – benefit amount
- **discrete** – whether annuity due (True) or continuous (False)
- **variance** – whether first moment (False) or variance (True)

Examples

```
>>> Recursion().set_interest(i=0.06).set_a(7, x=1)
```

whole_life_annuity(*x: int*, *s: int = 0*, *b: int = 1*, *variance: bool = False*, *discrete: bool = True*) → float

Compute whole life annuity a_x by attempting recursion then twin first

Parameters

- **x** – age of selection
- **s** – years after selection
- **b** – annuity benefit amount
- **variance** (*bool*) – return EPV (True) or variance (False)
- **discrete** – annuity due (True) or continuous (False)

temporary_annuity(*x: int*, *s: int = 0*, *t: int = -999*, *b: int = 1*, *variance: bool = False*, *discrete: bool = True*) → float

Compute temporary annuity a_x:t by attempting recursion then twin first

Parameters

- **x** – age of selection

- **s** – years after selection
- **t** – term of annuity in years
- **b** – annuity benefit amount
- **variance** (*bool*) – return EPV (True) or variance (False)
- **discrete** – annuity due (True) or continuous (False)

deferred_annuity(*x: int*, *s: int = 0*, *t: int = -999*, *u: int = 0*, *b: int = 1*, *discrete: bool = True*) → float

Compute deferred annuity u|a_x:t by attempting recursion first

Parameters

- **x** – age of selection
- **s** – years after selection
- **u** – years deferred
- **t** – term of annuity in years
- **b** – annuity benefit amount
- **discrete** – annuity due (True) or continuous (False)

2.13 lifetable

Life Tables - Loads and calculates life tables

class actuarialmath.lifetable.**LifeTable**(*udd: bool = True*, *verbose: bool = False*, ***kwargs*)

Bases: *Reserves*

Calculate life table, and iteratively fill in missing values

Parameters

- **udd** – assume UDD or constant force of mortality for fractional ages
- **verbose** – whether to echo update steps

Notes

4 types of columns can be loaded and calculated in the life table:

- 'q' : probability (x) dies in one year
- 'l' : number of lives aged x
- 'd' : number of deaths of age x
- 'p' : probability (x) survives at least one year

set_table(*radix: int = 100000*, *minage: int = -1*, *maxage: int = -1*, *fill: bool = True*, *l: Dict[int, float] | None = None*, *d: Dict[int, float] | None = None*, *p: Dict[int, float] | None = None*, *q: Dict[int, float] | None = None*) → *LifeTable*

Update life table

Parameters

- **l** – lives at start of year x, or
- **d** – deaths in year x, or
- **p** – probabilities that (x) survives one year, or
- **q** – probabilities that (x) dies in one year
- **fill** – whether to automatically fill table cells (default is True)
- **minage** – minimum age in table
- **maxage** – maximum age in table
- **radix** – initial number of lives

Examples:

```
>>> life = LifeTable(udd=True).set_table(l={90: 1000, 93: 825},
>>>                                      d={97: 72},
>>>                                      p={96: .2},
>>>                                      q={95: .4, 97: 1})
```

fill_table(*radix: int*) → *LifeTable*

Iteratively fill in missing table cells (does not check consistency)

Parameters
radix – initial number of lives

mu_x(*x: int*, *s: int = 0*, *t: int = 0*) → float

Compute mu_x from p_x in life table

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – death within next t years

l_x(*x: int*, *s: int = 0*) → float

Lookup l_x from life table

Parameters

- **x** – age of selection
- **s** – years after selection

d_x(*x: int*, *s: int = 0*, *t: int = 1*) → float

Compute deaths as lives at x_t divided by lives at x

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – death within next t years

p_x(*x: int*, *s: int = 0*, *t: int = 1*) → float

t_p_x = lives beginning year x+t divided lives beginning year x

Parameters

- **x** – age of selection

- **s** – years after selection
- **t** – death within next t years

q_x(*x: int*, *s: int = 0*, *t: int = 1*, *u: int = 0*) → float

Deferred mortality: u|t_q_x = (l[x+u] - l[x+u+t]) / l[x]

Parameters

- **x** – age of selection
- **s** – years after selection
- **u** – survive u years, then…
- **t** – death within next t years

e_x(*x: int*, *s: int = 0*, *n: int = -999*, *curtate: bool = True*, *moment: int = 1*) → float

Expected curtate lifetime from sum of lives in table

Parameters

- **x** – age of selection
- **s** – years after selection
- **n** – future lifetime limited at n years
- **curtate** – whether curtate (True) or complete (False) expectations
- **moment** – whether to compute first (1) or second (2) moment

E_x(*x: int*, *s: int = 0*, *t: int = 1*, *moment: int = 1*) → float

Pure Endowment from life table and interest rate

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – survives t years
- **moment** – return first (1) or second (2) moment or variance (-2)

__getitem__(*col: str*) → Dict[int, float]

Returns a column of the life table

Parameters

col – name of life table column to return

frame() → DataFrame

Return life table columns and values in a DataFrame

2.14 sult

SULT - Loads and uses a standard ultimate life table

class actuarialmath.sult.**SULT**(*i: float = 0.05, radix: int = 100000, S: ~typing.Callable[[float, float], float] = <function _faml_sult>, minage: int = 20, maxage: int = 130, **kwargs*)

Bases: *LifeTable*

Generates and uses a standard ultimate life table

Parameters

- **i** – interest rate
- **radix** – initial number of lives
- **minage** – minimum age
- **maxage** – maximum age
- **S** – survival function, default is Makeham with SOA FAM-L parameters

Examples

```
>>> sult = SULT()
>>> a = sult.temporary_annuity(70, t=10)
>>> A = sult.deferred_annuity(70, u=10)
>>> P = sult.gross_premium(a=a, A=A, benefit=100000, initial_premium=0.75,
>>>                        renewal_premium=0.05)
```

__getitem__(*col: str*) → Dict[int, float]

Returns a column of the sult table

Parameters

col – name of life table column to return

frame(*minage: int = 20, maxage: int = 100*)

Derive FAM-L exam table columns of SULT as a DataFrame

Parameters

- **minage** – first age to display row
- **maxage** – large age to display row

2.15 selectlife

Select and Ultimate Life Table – Loads and calculates select life tables

MIT License. Copyright 2022-2023 Terence Lim

class actuarialmath.selectlife.**SelectLife**(*periods: int = 0, udd: bool = True, verbose: bool = False, **kwargs*)

Bases: *LifeTable*

Calculate select life table, and iteratively fill in missing values

Parameters

- **periods** – number of select period years
- **verbose** – whether to echo update steps

Notes

6 types of columns can be loaded and calculated in the select table:

- 'q' : probability [x]+s dies in one year
- 'l' : number of lives aged [x]+s
- 'd' : number of deaths of age [x]+s
- 'A' : whole life insurance
- 'a' : whole life annuity
- 'e' : expected future curtate lifetime of [x]+s

__getitem__(*table: str*) → Dict[int, float]

Returns values from a select and ultimate table

Parameters

table – may be {'l', 'q', 'e', 'd', 'a', 'A'}

set_table(*l: Dict[int, List[float]] | None = None*, *d: Dict[int, List[float]] | None = None*, *q: Dict[int, List[float]] | None = None*, *A: Dict[int, List[float]] | None = None*, *a: Dict[int, List[float]] | None = None*, *e: Dict[int, List[float]] | None = None*, *fill: bool = True*, *radix: int = 100000*) → *SelectLife*

Update from table, every age has row for all select durations

Parameters

- **q** – probability [x]+s dies in one year
- **l** – number of lives aged [x]+s
- **d** – number of deaths of [x]+s
- **A** – whole life insurance, or
- **a** – whole life annuity, or
- **e** – expected future lifetime of [x]+s
- **radix** – initial number of lives
- **fill** – whether to fill missing values using recursion formulas

Examples

```
>>> life = SelectLife().set_table(l={55: [10000, 9493, 8533, 7664],
>>>                                  56: [8547, 8028, 6889, 5630],
>>>                                  57: [7011, 6443, 5395, 3904],
>>>                                  58: [5853, 4846, 3548, 2210]},
>>>                               e={57: [None, None, None, 1]})
>>> print(life.e_r(58, s=2))
```

set_select(*s: int*, *age_selected: bool*, *radix: int = 100000*, *fill: bool = False*, *l: Dict[int, float] | None = None*, *d: Dict[int, float] | None = None*, *q: Dict[int, float] | None = None*, *A: Dict[int, float] | None = None*, *a: Dict[int, float] | None = None*, *e: Dict[int, float] | None = None*) → *SelectLife*

Update a table column, for a particular duration s in the select period

Parameters

- **s** – column to populate - n is ultimate, 0..n-1 is year after select
- **age_selected** – is indexed by age selected or actual (False, default)
- **radix** – initial number of lives
- **fill** – whether to fill missing values using recursion formulas
- **q** – probabilities [x]+s dies in next year, by age
- **l** – number of lives aged [x]+s, by age
- **d** – number of deaths of [x]+s, by age
- **A** – whole life insurance of [x]+s, by age
- **a** – whole life annuity of [x]+s, by age
- **e** – expected future lifetime of [x]+s, by age

fill_table(*radix: int = 100000*) → *SelectLife*

Fills in missing table values (does not check for consistency)

Parameters

radix – initial number of lives

A_x(*x: int*, *s: int = 0*, *moment: int = 1*, *discrete: bool = True*, ***kwargs*) → float

Returns insurance value computed from select table

Parameters

- **x** – age of selection
- **s** – years after selection

a_x(*x: int*, *s: int = 0*, *moment: int = 1*, *discrete: bool = True*, ***kwargs*) → float

Returns annuity value computed from select table

Parameters

- **x** – age of selection
- **s** – years after selection

l_x(*x: int*, *s: int = 0*) → float

Returns number of lives aged [x]+s computed from select table

Parameters

- **x** – age of selection
- **s** – years after selection

p_x(*x: int*, *s: int = 0*, *t: int = 1*) → float

t_p_[x]+s by chain rule: prod(1_p_[x]+s+y) for y in range(t)

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – survives t years

q_x(*x: int*, *s: int = 0*, *t: int = 1*, *u: int = 0*) → float

t|u_q_[x]+s = [x]+s survives u years, does not survive next t

Parameters

- **x** – age of selection
- **s** – years after selection
- **u** – survives u years, then
- **t** – dies within next t years

e_x(*x: int*, *s: int = 0*, *t: int = -999*, *curtate: int = True*) → float

Returns expected life time computed from select table

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – limit of expected future lifetime

frame(*table: str = 'l'*) → DataFrame

Returns select and ultimate table values as a DataFrame

Parameters

table – table to return, one of [‘A’, ‘a’, ‘q’, ‘d’, ‘e’, ‘l’]

Examples

```
>>> table={21: [.00120, .00150, .00170, .00180],
>>>        22: [.00125, .00155, .00175, .00185],
>>>        23: [.00130, .00160, .00180, .00195]}
>>> life = SelectLife(verbose=True).set_table(q=table)
>>> print(life.frame('l').round(1))
>>> print(life.frame('q').round(6))
```

2.16 mortalitylaws

Mortality Laws: Applies shortcut formulas for Beta, Uniform, Makeham and Gompertz

class actuarialmath.mortalitylaws.**MortalityLaws**(***kwargs*)

Bases: *Reserves*

Apply shortcut formulas for special mortality laws

l_r(*x: int*, *s: int = 0*, *r: float = 0.0*) → float

Fractional lives given special mortality law: l_[x]+s+r

Parameters

- **x** – age of selection
- **s** – years after selection
- **r** – fractional year after selection

p_r(*x: int*, *s: int = 0*, *r: float = 0.0*, *t: float = 1.0*) → float

Fractional age survival probability given special mortality law

Parameters

- **x** – age of selection
- **s** – years after selection
- **r** – fractional year after selection
- **t** – death within next t fractional years

q_r(*x: int*, *s: int = 0*, *r: float = 0.0*, *t: float = 1.0*, *u: float = 0.0*) → float

Fractional age deferred mortality given special mortality law

Parameters

- **x** – age of selection
- **s** – years after selection
- **r** – fractional year after selection
- **u** – survive u fractional years, then…
- **t** – death within next t fractional years

mu_r(*x: int*, *s: int = 0*, *r: float = 0.0*) → float

Fractional age force of mortality given special mortality law

Parameters

- **x** – age of selection
- **s** – years after selection
- **r** – fractional year after selection

f_r(*x: int*, *s: int = 0*, *r: float = 0.0*, *t: float = 0.0*) → float

fractional age lifetime density given special mortality law

Parameters

- **x** – age of selection
- **s** – years after selection
- **r** – fractional year after selection
- **t** – mortality at fractional year t

e_r(*x: int*, *s: int = 0*, *r: float = 0.0*, *t: float = -999*) → float

Fractional age future lifetime given special mortality law

Parameters

- **x** – age of selection
- **s** – years after selection
- **r** – fractional year after selection
- **t** – limited at t years

class actuarialmath.mortalitylaws.**Beta**(*omega: int*, *alpha: float*, *radix: int = 100000*, ***kwargs*)

Bases: *MortalityLaws*

Shortcuts with beta distribution of deaths (is Uniform when alpha = 1)

Parameters

- **omega** – maximum age
- **alpha** – alpha paramter of beta distribution
- **radix** – assumed starting number of lives for survival function

Examples

```
>>> print(Beta(omega=60, alpha=1/3).mu_x(35) * 1000)
```

e_r(*x: int*, *s: int = 0*, *t: float = -999*) → float

Expectation of future lifetime through fractional age: e_[x]+s:t

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – limited at t years

e_x(*x: int*, *s: int = 0*, *n: int = -999*, *curtate: bool = False*, *moment: int = 1*) → float

Shortcut formula for complete expectation

Parameters

- **x** – age of selection
- **s** – years after selection
- **n** – death within next n fractional years
- **t** – limited at t years
- **curtate** – whether curtate (True) or complete (False) lifetime
- **moment** – whether to compute first (1) or second (2) moment

class actuarialmath.mortalitylaws.**Uniform**(*omega: int*, ***kwargs*)

Bases: *Beta*

Shortcuts with uniform distribution of deaths aka DeMoivre's Law

Parameters

omega – maximum age

Examples

```
>>> print(Uniform(95).e_x(30, t=40, curtate=False)) # 27.692
```

e_x(*x: int*, *s: int = 0*, *t: int = -999*, *curtate: bool = False*, *moment: int = 1*) → float

Shortcut for expected future lifetime

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – limited at t years
- **curtate** – whether curtate (True) or complete (False) lifetime
- **moment** – whether to compute first (1) or second (2) moment

E_x(*x: int*, *s: int = 0*, *t: int = -999*, *moment: int = 1*) → float

Shortcut for Pure Endowment

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – term of pure endowment
- **endowment** – amount of pure endowment
- **moment** – compute first or second moment

whole_life_insurance(*x: int*, *s: int = 0*, *moment: int = 1*, *b: int = 1*, *discrete: bool = True*) → float

Shortcut for whole life insurance

Parameters

- **x** – age of selection
- **s** – years after selection
- **b** – amount of benefit
- **moment** – compute first or second moment
- **discrete** – benefit paid year-end (True) or moment of death (False)

term_insurance(*x: int*, *s: int = 0*, *t: int = 1*, *b: int = 1*, *moment: int = 1*, *discrete: bool = False*) → float

Shortcut for term insurance

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – term of insurance
- **b** – amount of benefit
- **moment** – compute first or second moment
- **discrete** – benefit paid year-end (True) or moment of death (False)

class actuarialmath.mortalitylaws.**Makeham**(*A: float*, *B: float*, *c: float*, ***kwargs*)

Bases: *MortalityLaws*

Includes element in force of mortality that does not depend on age

Parameters

- **A** – parameters of Makeham distribution
- **B** – parameters of Makeham distribution
- **c** – parameters of Makeham distribution

Examples

```
>>> print(Makeham(A=0.00022, B=2.7e-6, c=1.124).mu_x(60) * 0.9803)  # 0.00316
```

class actuarialmath.mortalitylaws.**Gompertz**(*B: float*, *c: float*, ***kwargs*)

Bases: *Makeham*

Is Makeham's Law with A = 0

Parameters

- **B** – parameters of Gompertz distribution
- **c** – parameters of Gompertz distribution

Examples

```
>>> print(Gompertz(B=0.00027, c=1.1).f_x(50, t=10)) # 0.04839
```

2.17 constantforce

Constant Force of Mortality - shortcut formulas

class actuarialmath.constantforce.**ConstantForce**(*mu: float*, ***kwargs*)

Bases: *MortalityLaws*

Constant force of mortality - memoryless exponential distribution of lifetime

Parameters

mu – constant value of force of mortality

Examples

```
>>> life = ConstantForce(mu=0.01).set_interest(delta=0.05)
>>> life.term_insurance(35, t=35, discrete=False) + life.E_x(35, t=35)*0.51791
```

e_x(*x: int*, *s: int = 0*, *t: int = -999*, *curtate: bool = False*, *moment: int = 1*) → float

Expected lifetime E[T_x] is memoryless: does not depend on (x)

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – limited at year t
- **curtate** – whether curtate (True) or continuous (False) lifetime
- **moment** – first (1) or second (2) moment

E_x(*x: int*, *s: int = 0*, *t: int = -999*, *endowment: int = 1*, *moment: int = 1*) → float

Shortcut for pure endowment: does not depend on age x

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – term of pure endowment
- **endowment** – amount of pure endowment
- **moment** – compute first or second moment

whole_life_annuity(*x: int*, *s: int = 0*, *b: int = 1*, *variance: bool = False*, *discrete: bool = True*) → float

Shortcut for whole life annuity: does not depend on age x

Parameters

- **x** – age of selection
- **s** – years after selection
- **b** – annuity benefit amount
- **variance** – return APV (True) or variance (False)
- **discrete** – annuity due (True) or continuous (False)

temporary_annuity(*x: int*, *s: int = 0*, *t: int = -999*, *b: int = 1*, *variance: bool = False*, *discrete: bool = True*) → float

Shortcut for temporary life annuity: does not depend on age x

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – term of annuity in years
- **b** – annuity benefit amount
- **variance** – return APV (True) or variance (False)
- **discrete** – annuity due (True) or continuous (False)

whole_life_insurance(*x: int*, *s: int = 0*, *moment: int = 1*, *b: int = 1*, *discrete: bool = True*) → float

Shortcut for APV of whole life: does not depend on age x

Parameters

- **x** – age of selection
- **s** – years after selection
- **b** – amount of benefit
- **moment** – compute first or second moment
- **discrete** – benefit paid year-end (True) or moment of death (False)

term_insurance(*x: int*, *s: int = 0*, *t: int = 1*, *b: int = 1*, *moment: int = 1*, *discrete: bool = True*) → float

Shortcut for APV of term life: does not depend on age x

Parameters

- **x** – age of selection
- **s** – years after selection
- **t** – term of insurance
- **b** – amount of benefit
- **moment** – compute first or second moment
- **discrete** – benefit paid year-end (True) or moment of death (False)

Z_t(*x: int*, *prob: float*, *discrete: bool = True*) → float

Shortcut for T_x (or K_x) given survival probability for insurance

Parameters

- **x** – age (does not depend)
- **prob** – desired probability threshold
- **discrete** – benefit paid year-end (True) or moment of death (False)

Y_t(*x: int*, *prob: float*, *discrete: bool = True*) → float

Shortcut for T_x (or K_x) given survival probability for annuity

Parameters

- **x** – age (does not depend)
- **prob** – desired probability threshold
- **discrete** – continuous (False) or annuity due (True)

2.18 extrarisk

Extra Risk - Adjusts force of mortality, age rating or mortality rate

class actuarialmath.extrarisk.**ExtraRisk**(*life:* Survival, *risk: str = ''*, *extra: float = 0.0*)

Bases: *Actuarial*

Adjust mortality by extra risk

Parameters

- **life** – contains original survival and mortality rates
- **extra** – amount of extra risk to adjust
- **risk** – adjust by {"ADD_FORCE", "MULTIPLY_FORCE", "ADD_AGE", "MULTIPLY_RATE"}

risks = ['ADD_FORCE', 'MULTIPLY_FORCE', 'ADD_AGE', 'MULTIPLY_RATE']

__getitem__(*col: str*) → Dict[int, float]

Returns survival function values adjusted by extra risk

Parameters

col – {'p', 'q'} for one-year survival or mortality function values

Returns

dict of age and survival function values adjusted by extract risk

Examples

```
>>> life = SULT()
>>> extra = ExtraRisk(life=life, extra=0.05, risk="ADD_FORCE")
>>> select = SelectLife(periods=1).set_select(s=0, age_selected=True,
                                              q=extra['q'])
```

p_x(*x: int*, *s: int = 0*) → float

Return p_[x]+s after adding or multiplying force of mortality

Parameters

- **x** – age of selection
- **s** – years after selection

Examples

```
>>> life = SULT()
>>> extra = ExtraRisk(life=life, extra=2, risk="MULTIPLY_FORCE")
>>> print(life.p_x(45), extra.p_x(45))
```

q_x(*x: int*, *s: int = 0*) → float

Return q_[x]+s after adding age rating or multipliying mortality rate

Parameters

- **x** – age of selection
- **s** – years after selection

2.19 mthly

1/Mthly - Calculates m'thly-pay insurance and annuities

class actuarialmath.mthly.**Mthly**(*m: int*, *life:* Annuity)

Bases: *Actuarial*

Compute 1/M'thly insurance and annuities

Parameters

- **m** – number of payments per year
- **life** – original survival and life contingent functions

v_m(*k: int*) → float

Compute discount rate compounded over k m'thly periods

Parameters
k – number of m'thly periods to compound

q_m(*x: int*, *s_m: int = 0*, *t_m: int = 1*, *u_m: int = 0*) → float

Compute deferred mortality over m'thly periods

Parameters

- **x** – year of selection
- **s_m** – number of m'thly periods after selection
- **u_m** – survive number of m'thly periods , then
- **t_m** – dies within number of m'thly periods

p_m(*x: int*, *s_m: int = 0*, *t_m: int = 1*) → float

Compute survival probability over m'thly periods

Parameters

- **x** – year of selection
- **s_m** – number of m'thly periods after selection
- **t_m** – survives number of m'thly periods

Z_m(*x: int*, *s: int = 0*, *t: int = 1*, *benefit: ~typing.Callable = <function Mthly.<lambda>>*, *moment: int = 1*)

Return PV of insurance r.v. Z and probability of death at mthly intervals

Parameters

- **x** – year of selection
- **s** – years after selection
- **t** – year of death
- **benefit** – amount of benefit by year and age selected
- **moment** – return first or second moment

Returns
DataFrame, indexed by mthly period, with column names ['Z', 'p']

Examples

```
>>> life = LifeTable(udd=False).set_table(q={0:.16,1:.23})          >>>
↪                    .set_interest(i_m=.18,m=2)
>>> mthly = Mthly(m=2, life=life)
>>> Z = mthly.Z_m(0, t=2, benefit=lambda x,t: 300000 + t*30000*2)
```

E_x(*x: int*, *s: int = 0*, *t: int = 1*, *moment: int = 1*, *endowment: int = 1*) → float

Compute pure endowment factor

Parameters

- **x** – year of selection
- **s** – years after selection
- **t** – term length in years
- **moment** – return first or second moment
- **endowment** – endowment amount

A_x(*x: int*, *s: int = 0*, *t: int = 1*, *u: int = 0*, *benefit: ~typing.Callable = <function Mthly.<lambda>>*, *moment: int = 1*) → float

Compute insurance factor with m'thly benefits

Parameters

- **x** – year of selection
- **s** – years after selection
- **u** – years deferred
- **t** – term of insurance in years
- **benefit** – amount of benefit by year and age selected
- **moment** – return first or second moment

whole_life_insurance(*x: int*, *s: int = 0*, *moment: int = 1*, *b: int = 1*) → float

Whole life insurance: A_x

Parameters

- **x** – age of selection
- **s** – years after selection
- **b** – amount of benefit
- **moment** – compute first or second moment

term_insurance(*x: int*, *s: int = 0*, *t: int = 1*, *b: int = 1*, *moment: int = 1*) → float

Term life insurance: A_x:t^1

Parameters

- **x** – year of selection
- **s** – years after selection
- **t** – term of insurance in years
- **b** – amount of benefit

- **moment** – return first or second moment

deferred_insurance(*x: int*, *s: int = 0*, *n: int = 0*, *b: int = 1*, *t: int = -999*, *moment: int = 1*) → float

Deferred insurance n|_A_x:t^1 = discounted whole life

Parameters

- **x** – year of selection
- **s** – years after selection
- **u** – years to defer
- **t** – term of insurance in years
- **b** – amount of benefit
- **moment** – return first or second moment

endowment_insurance(*x: int*, *s: int = 0*, *t: int = 1*, *b: int = 1*, *endowment: int = -1*, *moment: int = 1*) → float

Endowment insurance: A_x:t = term insurance + pure endowment

Parameters

- **x** – year of selection
- **s** – years after selection
- **t** – term of insurance in years
- **b** – amount of benefit
- **endowment** – amount of endowment
- **moment** – return first or second moment

insurance_twin(*a: float*) → float

Return insurance twin of m'thly annuity

Parameters
a – twin annuity factor

annuity_twin(*A: float*) → float

Return value of annuity twin of m'thly insurance

Parameters
A – amount of m'thly insurance

Examples

```
>>> mthly = Mthly(m=12, life=Annuity().set_interest(i=0.06))
>>> mthly.annuity_twin(A=0.4075)*15*12
```

annuity_variance(*A2: float*, *A1: float*, *b: float = 1*) → float

Variance of m'thly annuity from m'thly insurance moments

Parameters

- **A2** – double force of interest of m'thly insurance
- **A1** – first moment of m'thly insurance
- **b** – amount of benefit

Examples

```
>>> mthly = Mthly(m=12, life=Annuity().set_interest(i=0.06))
>>> mthly.annuity_variance(A1=0.4075, A2=0.2105, b=15*12)
```

whole_life_annuity(*x: int*, *s: int = 0*, *b: int = 1*, *variance: bool = False*) → float

Whole life m'thly annuity: a_x

Parameters

- **x** – year of selection
- **s** – years after selection
- **b** – amount of benefit
- **variance** – return first moment (False) or variance (True)

temporary_annuity(*x: int*, *s: int = 0*, *t: int = -999*, *b: int = 1*, *variance: bool = False*) → float

Temporary m'thly life annuity: a_x:t

Parameters

- **x** – year of selection
- **s** – years after selection
- **t** – term of annuity in years
- **b** – amount of benefit
- **variance** – return first moment (False) or variance (True)

deferred_annuity(*x: int*, *s: int = 0*, *u: int = 0*, *t: int = -999*, *b: int = 1*) → float

Deferred m'thly life annuity due n|t_a_x = n+t_a_x - n_a_x

Parameters

- **x** – year of selection
- **s** – years after selection
- **u** – years of deferral
- **t** – term of annuity in years
- **b** – amount of benefit

immediate_annuity(*x: int*, *s: int = 0*, *t: int = -999*, *b: int = 1*) → float

Immediate m'thly annuity

Parameters

- **x** – year of selection
- **s** – years after selection
- **t** – term of annuity in years
- **b** – amount of benefit

2.20 udd

UDD - 1/mthly insurance and annuities with uniform distribution of deaths

class actuarialmath.udd.**UDD**(*m: int*, *life:* Annuity, ***kwargs*)

Bases: *Mthly*

1/mthly shortcuts with UDD assumption

Parameters

- **m** – number of payments per year
- **life** – original fractional survival and mortality functions

static alpha(*m: int*, *i: float*) → float

Compute 1/mthly UDD interest rate beta function value

Parameters

- **m** – number of payments per year
- **i** – annual interest rate

static beta(*m: int*, *i: float*) → float

Compute 1/mthly UDD interest rate alpha function value

Parameters

- **m** – number of payments per year
- **i** – annual interest rate

whole_life_insurance(*x: int*, *s: int = 0*, *moment: int = 1*, *b: int = 1*) → float

1/mthly UDD Whole life insurance: A_x

Parameters

- **x** – age of selection
- **s** – years after selection
- **b** – amount of benefit
- **moment** – compute first or second moment

Examples

```
>>> UDD(m=0, life=SULT(udd=True)).whole_life_insurance(x)
```

endowment_insurance(*x: int*, *s: int = 0*, *t: int = 1*, *b: int = 1*, *endowment: int = -1*, *moment: int = 1*) → float

1/mthly UDD Endowment insurance = term insurance + pure endowment

Parameters

- **x** – year of selection
- **s** – years after selection
- **t** – term of insurance in years

- **b** – amount of benefit
- **endowment** – amount of endowment
- **moment** – return first or second moment

term_insurance(*x: int*, *s: int = 0*, *t: int = 1*, *b: int = 1*, *moment: int = 1*) → float

1/mthly UDD Term insurance: A_x:t

Parameters

- **x** – year of selection
- **s** – years after selection
- **t** – term of insurance in years
- **b** – amount of benefit
- **moment** – return first or second moment

Examples

```
>>> UDD(m=12, life=SULT(udd=True)).term_insurance(45)
```

whole_life_annuity(*x: int*, *s: int = 0*, *b: int = 1*, *variance: bool = False*) → float

1/mthly UDD Whole life annuity: a_x

Parameters

- **x** – year of selection
- **s** – years after selection
- **b** – amount of benefit
- **variance** – return first moment (False) or variance (True)

Examples

```
>>> UDD(m=12, life=SULT(udd=True)).whole_life_annuity(x)
```

temporary_annuity(*x: int*, *s: int = 0*, *t: int = -999*, *b: int = 1*, *variance: bool = False*) → float

1/mthly UDD Temporary life annuity: a_x:t

Parameters

- **x** – year of selection
- **s** – years after selection
- **t** – term of annuity in years
- **b** – amount of benefit
- **variance** – return first moment (False) or variance (True)

Examples

```
>>> UDD(m=12, life=SULT(udd=True)).temporary_annuity(45, t=20)
```

deferred_annuity(*x: int*, *s: int = 0*, *u: int = 0*, *t: int = -999*, *b: int = 1*, *variance: bool = False*) → float

1/mthly UDD Deferred life annuity n|t_a_x = n+t_a_x - n_a_x

Parameters

- **x** – year of selection
- **s** – years after selection
- **u** – years of deferral
- **t** – term of annuity in years
- **b** – amount of benefit

static interest_frame(*i: float = 0.05*)

Return 1/mthly UDD interest function values in a DataFrame

Parameters

i – annual interest rate

2.21 woolhouse

Woolhouse - 1/mthly insurance and annuities with Woolhouse's approximation

MIT License. Copyright 2022-2023 Terence Lim

class actuarialmath.woolhouse.**Woolhouse**(*m: int*, *life: Annuity*, *three_term: bool = False*, *approximate_mu: Callable[[int, int], float] | bool = True*)

Bases: *Mthly*

1/m'thly shortcuts with Woolhouse approximation

Parameters

- **m** – number of payments per year
- **life** – original fractional survival and mortality functions
- **three_term** – whether to include (True) or ignore (False) third term
- **approximate_mu** – exact (False), approximate (True) or function for third term

mu_x(*x: int*, *s: int = 0*) → float

Computes mu_x or calls approximate_mu for third term

Parameters

- **x** – age of selection
- **s** – years after selection

whole_life_insurance(*x: int*, *s: int = 0*, *b: int = 1*, *mu: float | None = 0.0*) → float

1/m'thly Woolhouse whole life insurance: A_x

Parameters

- **x** – age of selection

- **s** – years after selection
- **b** – amount of benefit
- **mu** – value of mu at age x+s

term_insurance(*x: int*, *s: int = 0*, *t: int = -999*, *b: int = 1*, *mu: float | None = 0.0*, *mu1: float | None = None*) → float

1/m'thly Woolhouse term insurance: A_x:t

Parameters

- **x** – year of selection
- **s** – years after selection
- **t** – term of insurance in years
- **b** – amount of benefit
- **mu** – value of mu at age x+s
- **mu1** – value of mu at age x+s+t

endowment_insurance(*x: int*, *s: int = 0*, *t: int = -999*, *b: int = 1*, *endowment: int = -1*, *mu: float | None = None*) → float

1/m'thly Woolhouse term insurance: A_x:t

Parameters

- **x** – year of selection
- **s** – years after selection
- **t** – term of insurance in years
- **b** – amount of benefit
- **endowment** – amount of endowment
- **mu** – value of mu at age x+s+t

deferred_insurance(*x: int*, *s: int = 0*, *u: int = 0*, *t: int = -999*, *b: int = 1*, *mu: float | None = None*, *mu1: float | None = None*) → float

1/m'thly Woolhouse deferred insurance as discounted term or WL

Parameters

- **x** – year of selection
- **s** – years after selection
- **u** – number of years deferred
- **t** – term of insurance in years
- **b** – amount of benefit
- **mu** – value of mu at age x+s+u
- **mu1** – value of mu at age x+s+u+t

whole_life_annuity(*x: int*, *s: int = 0*, *b: int = 1*, *mu: float | None = None*) → float

1/m'thly Woolhouse whole life annuity: a_x

Parameters

- **x** – year of selection

- **s** – years after selection
- **b** – amount of benefit
- **mu** – value of mu at age x+s

Examples

```
>>> life = Recursion().set_interest(i=0.05).set_a(3.4611, x=0)
>>> Woolhouse(m=4, life=life).whole_life_annuity(x=0)
```

temporary_annuity(*x: int*, *s: int = 0*, *t: int = -999*, *b: int = 1*, *mu: float | None = None*, *mu1: float | None = None*) → float

1/m'thly Woolhouse temporary life annuity: a_x

Parameters

- **x** – year of selection
- **s** – years after selection
- **t** – term of annuity in years
- **b** – amount of benefit
- **mu** – value of mu at age x+s
- **mu1** – value of mu at age x+s+t

deferred_annuity(*x: int*, *s: int = 0*, *t: int = -999*, *u: int = 0*, *b: int = 1*, *mu: float | None = None*, *mu1: float | None = None*) → float

1/m'thly Woolhouse deferred life annuity: a_x

Parameters

- **x** – year of selection
- **s** – years after selection
- **u** – years of deferral
- **t** – term of annuity in years
- **mu** – value of mu at age x+s+u
- **mu1** – value of mu at age x+s+u+t

CHAPTER

THREE

INDICES AND TABLES

- genindex
- modindex
- search

PYTHON MODULE INDEX

a

INDEX

Symbols

A

B

C

D

E

F

G

I

L

M

N

P

Q

R

S

T

U

V

W

Y

Z